📖 Chapter 5: React.js

Table of Contents

5.1 What is React?

> ***Definition:*** *React is a JavaScript library for building user interfaces using reusable components. It uses a virtual DOM for efficient updates and a declarative approach where you describe what the UI should look like, and React efficiently handles the rendering and updates.*

Key Features:

- Component-based architecture
- Virtual DOM for performance
- Declarative UI
- One-way data flow
- Rich ecosystem (Router, Redux, etc.)

5.2 Components

> ***Definition:*** *Components are independent, reusable pieces of UI that accept inputs (props) and return React elements describing what should appear on screen. Modern React uses functional components with hooks instead of class components.*

```
import React from 'react';

// ==================== FUNCTIONAL COMPONENT ====================
function Welcome({ name }) {
    return <h1>Hello, {name}!</h1>;
}

// Arrow function syntax
const Welcome = ({ name }) => {
    return <h1>Hello, {name}!</h1>;
```

```
};

// Implicit return (for simple components)
const Welcome = ({ name }) => <h1>Hello, {name}!</h1>;

// ==================== USING COMPONENTS ====================
function App() {
    return (
        <div>
            <Welcome name="John" />
            <Welcome name="Jane" />
            <Welcome name="Bob" />
        </div>
    );
}

// ==================== COMPONENT COMPOSITION ====================
function Card({ title, children }) {
    return (
        <div className="card">
            <div className="card-header">
                <h2>{title}</h2>
            </div>
            <div className="card-body">
                {children}
            </div>
        </div>
    );
}

function App() {
    return (
        <Card title="User Profile">
            <p>Name: John Doe</p>
            <p>Email: john@example.com</p>
            <button>Edit Profile</button>
        </Card>
    );
}

// ==================== EXPORT PATTERNS ====================
// Default export (one per file)
export default function MyComponent() { }

// Named export (multiple per file)
export function ComponentA() { }
export function ComponentB() { }

// Import
import MyComponent from './MyComponent';
import { ComponentA, ComponentB } from './Components';
```

5.3 JSX

> ***Definition:*** *JSX (JavaScript XML) is a syntax extension that allows writing HTML-like code in JavaScript. It gets compiled to* `React.createElement()` *calls by Babel. JSX makes component templates more readable and allows embedding JavaScript expressions using curly braces* `{}` *.*

```
// ==================== BASIC JSX ====================
const element = <h1>Hello, World!</h1>;

// Compiles to:
const element = React.createElement('h1', null, 'Hello, World!');

// ==================== EMBEDDING EXPRESSIONS ====================
const name = 'John';
const element = <h1>Hello, {name}!</h1>;

// Any JavaScript expression works
const element = <h1>2 + 2 = {2 + 2}</h1>;
const element = <h1>{user.name.toUpperCase()}</h1>;
const element = <h1>{formatDate(new Date())}</h1>;

// ==================== JSX RULES ====================

// 1. Must return single element (use fragment or div)
// ❌ Wrong
return (
    <h1>Title</h1>
    <p>Content</p>
);

// ✅ Correct - Wrap in div
return (
    <div>
        <h1>Title</h1>
        <p>Content</p>
    </div>
);

// ✅ Correct - Use Fragment (no extra DOM element)
return (
    <>
        <h1>Title</h1>
        <p>Content</p>
    </>
);

// Or explicit Fragment
import { Fragment } from 'react';
return (
    <Fragment>
        <h1>Title</h1>
```

```
        <p>Content</p>
    </Fragment>
);

// 2. className instead of class
<div className="container">

// 3. htmlFor instead of for (in labels)
<label htmlFor="email">Email</label>

// 4. camelCase for attributes
<input onClick={handleClick} onChange={handleChange} />
<div tabIndex={0} onKeyDown={handleKey} />

// 5. Self-closing tags
<img src="image.jpg" alt="Description" />
<input type="text" />
<br />

// 6. Style as object with camelCase
<div style={{
    backgroundColor: 'blue',
    fontSize: '16px',
    marginTop: '10px'
}}>

// ==================== CONDITIONAL RENDERING IN JSX ====================

// Ternary operator
{isLoggedIn ? <LogoutButton /> : <LoginButton />}

// Logical AND (short-circuit)
{isLoggedIn && <UserDashboard />}
{messages.length > 0 && <Badge count={messages.length} />}

// Null for nothing
{showMessage ? <Message /> : null}

// ==================== RENDERING ARRAYS ====================
const items = ['Apple', 'Banana', 'Orange'];

<ul>
    {items.map((item, index) => (
        <li key={index}>{item}</li>
    ))}
</ul>

// ==================== DANGEROUS HTML ====================
// Be careful - XSS vulnerability!
<div dangerouslySetInnerHTML={{ __html: htmlContent }} />
```

5.4 Props

> ***Definition:*** *Props (properties) are read-only inputs passed from parent to child components. They allow data to flow down the component tree (unidirectional data flow) and make components reusable by configuring them with different values.*

```
// ==================== PASSING PROPS ====================
function Parent() {
    const user = { name: 'John', age: 30 };

    return (
        <Child
            name="John"
            age={30}
            isActive={true}
            items={['a', 'b', 'c']}
            user={user}
            onClick={() => console.log('Clicked')}
        />
    );
}

// ==================== RECEIVING PROPS ====================

// Method 1: props object
function Child(props) {
    return (
        <div>
            <p>Name: {props.name}</p>
            <p>Age: {props.age}</p>
            {props.isActive && <span>Active</span>}
            <button onClick={props.onClick}>Click</button>
        </div>
    );
}

// Method 2: Destructuring (preferred)
function Child({ name, age, isActive, onClick }) {
    return (
        <div>
            <p>Name: {name}</p>
            <p>Age: {age}</p>
            {isActive && <span>Active</span>}
            <button onClick={onClick}>Click</button>
        </div>
    );
}

// ==================== DEFAULT PROPS ====================
function Child({ name = 'Guest', age = 0, role = 'User' }) {
    return <p>{name} ({role}): {age} years old</p>;
```

```
}

// ==================== CHILDREN PROP ====================
function Card({ title, children }) {
    return (
        <div className="card">
            <h2>{title}</h2>
            <div className="card-body">
                {children}
            </div>
        </div>
    );
}

// Usage
<Card title="My Card">
    <p>This content is passed as children</p>
    <button>Action</button>
</Card>

// ==================== SPREAD PROPS ====================
const userProps = {
    name: 'John',
    age: 30,
    email: 'john@example.com'
};

<UserProfile {...userProps} />

// Equivalent to:
<UserProfile
    name={userProps.name}
    age={userProps.age}
    email={userProps.email}
/>

// ==================== RENDER PROPS ====================
function MouseTracker({ render }) {
    const [position, setPosition] = useState({ x: 0, y: 0 });

    const handleMouseMove = (e) => {
        setPosition({ x: e.clientX, y: e.clientY });
    };

    return (
        <div onMouseMove={handleMouseMove}>
            {render(position)}
        </div>
    );
}

// Usage
```

```jsx
<MouseTracker
    render={({ x, y }) => (
        <p>Mouse is at ({x}, {y})</p>
    )}
/>

// ==================== PROPS VALIDATION (PropTypes) ====================
import PropTypes from 'prop-types';

function User({ name, age, email, isActive }) {
    return <div>{name}</div>;
}

User.propTypes = {
    name: PropTypes.string.isRequired,
    age: PropTypes.number,
    email: PropTypes.string.isRequired,
    isActive: PropTypes.bool,
    role: PropTypes.oneOf(['user', 'admin']),
    items: PropTypes.arrayOf(PropTypes.string),
    user: PropTypes.shape({
        id: PropTypes.number,
        name: PropTypes.string
    })
};

User.defaultProps = {
    isActive: true,
    role: 'user'
};
```

5.5 useState Hook

> ***Definition:*** *`useState` is a Hook that adds state management to functional components. It returns an array with the current state value and a setter function. When state changes, the component re-renders with the new value.*

```jsx
import { useState } from 'react';

// ==================== BASIC USAGE ====================
function Counter() {
    // Declare state: [currentValue, setterFunction] = useState(initialValue)
    const [count, setCount] = useState(0);

    return (
        <div>
            <p>Count: {count}</p>
            <button onClick={() => setCount(count + 1)}>+</button>
            <button onClick={() => setCount(count - 1)}>-</button>
            <button onClick={() => setCount(0)}>Reset</button>
        </div>
```

```
    );
}

// ==================== FUNCTIONAL UPDATES ====================
// Use when new state depends on previous state
function Counter() {
    const [count, setCount] = useState(0);

    const increment = () => {
        // ❌ May cause issues with stale state
        setCount(count + 1);

        // ✅ Always use functional update for safety
        setCount(prevCount => prevCount + 1);
    };

    // Multiple updates in one handler
    const incrementBy3 = () => {
        // ❌ This only increments by 1 (batched updates)
        setCount(count + 1);
        setCount(count + 1);
        setCount(count + 1);

        // ✅ This correctly increments by 3
        setCount(prev => prev + 1);
        setCount(prev => prev + 1);
        setCount(prev => prev + 1);
    };

    return <button onClick={increment}>Count: {count}</button>;
}

// ==================== MULTIPLE STATE VARIABLES ====================
function Form() {
    const [name, setName] = useState('');
    const [email, setEmail] = useState('');
    const [age, setAge] = useState(0);
    const [isSubscribed, setIsSubscribed] = useState(false);

    return (
        <form>
            <input
                value={name}
                onChange={(e) => setName(e.target.value)}
            />
            <input
                value={email}
                onChange={(e) => setEmail(e.target.value)}
            />
        </form>
    );
}
```

```jsx
// ==================== OBJECT STATE ====================
function UserForm() {
    const [user, setUser] = useState({
        name: '',
        email: '',
        age: 0,
        address: {
            city: '',
            country: ''
        }
    });

    // ❌ Wrong - Mutating state directly
    const handleChange = (e) => {
        user.name = e.target.value;  // Never do this!
        setUser(user);
    };

    // ✅ Correct - Create new object with spread
    const handleChange = (e) => {
        const { name, value } = e.target;
        setUser(prevUser => ({
            ...prevUser,
            [name]: value
        }));
    };

    // Updating nested object
    const handleCityChange = (e) => {
        setUser(prevUser => ({
            ...prevUser,
            address: {
                ...prevUser.address,
                city: e.target.value
            }
        }));
    };

    return (
        <input
            name="name"
            value={user.name}
            onChange={handleChange}
        />
    );
}

// ==================== ARRAY STATE ====================
function TodoList() {
    const [todos, setTodos] = useState([
        { id: 1, text: 'Learn React', completed: false }
```

```
    ]);

    // Add item
    const addTodo = (text) => {
        const newTodo = { id: Date.now(), text, completed: false };
        setTodos([...todos, newTodo]);
        // Or: setTodos(prev => [...prev, newTodo]);
    };

    // Remove item
    const removeTodo = (id) => {
        setTodos(todos.filter(todo => todo.id !== id));
    };

    // Update item
    const toggleTodo = (id) => {
        setTodos(todos.map(todo =>
            todo.id === id
                ? { ...todo, completed: !todo.completed }
                : todo
        ));
    };

    // Update specific field
    const updateTodoText = (id, newText) => {
        setTodos(todos.map(todo =>
            todo.id === id
                ? { ...todo, text: newText }
                : todo
        ));
    };

    return (
        <ul>
            {todos.map(todo => (
                <li key={todo.id}>
                    <span
                        style={{
                            textDecoration: todo.completed ? 'line-through' : 'none'
                        }}
                    >
                        {todo.text}
                    </span>
                    <button onClick={() => toggleTodo(todo.id)}>Toggle</button>
                    <button onClick={() => removeTodo(todo.id)}>Delete</button>
                </li>
            ))}
        </ul>
    );
}

// ==================== LAZY INITIAL STATE ====================
```

```
// For expensive computations, pass a function
const [state, setState] = useState(() => {
    const initialValue = expensiveComputation();
    return initialValue;
});
```

5.6 useEffect Hook

> ***Definition:*** `useEffect` *is a Hook for performing side effects in functional components—data fetching, subscriptions, timers, DOM manipulation. It runs after render and can optionally return a cleanup function. The dependency array controls when it re-runs.*

```
import { useState, useEffect } from 'react';

// ==================== RUNS AFTER EVERY RENDER ====================
useEffect(() => {
    console.log('Component rendered');
}); // No dependency array

// ==================== RUNS ONCE ON MOUNT ====================
useEffect(() => {
    console.log('Component mounted');
    fetchData();
}, []); // Empty dependency array

// ==================== RUNS WHEN DEPENDENCIES CHANGE ====================
useEffect(() => {
    console.log('userId changed:', userId);
    fetchUser(userId);
}, [userId]); // Runs when userId changes

// ==================== CLEANUP FUNCTION ====================
useEffect(() => {
    const subscription = someAPI.subscribe(data);

    // Cleanup runs before next effect and on unmount
    return () => {
        subscription.unsubscribe();
    };
}, []);

// ==================== DATA FETCHING ====================
function UserProfile({ userId }) {
    const [user, setUser] = useState(null);
    const [loading, setLoading] = useState(true);
    const [error, setError] = useState(null);

    useEffect(() => {
        const fetchUser = async () => {
            try {
```

```
                setLoading(true);
                setError(null);
                const response = await fetch(`/api/users/${userId}`);
                if (!response.ok) throw new Error('Failed to fetch');
                const data = await response.json();
                setUser(data);
            } catch (err) {
                setError(err.message);
            } finally {
                setLoading(false);
            }
        };

        fetchUser();
    }, [userId]); // Re-fetch when userId changes

    if (loading) return <p>Loading...</p>;
    if (error) return <p>Error: {error}</p>;
    if (!user) return <p>No user found</p>;

    return <div>{user.name}</div>;
}

// ==================== ABORT CONTROLLER (Cleanup Fetch) ====================
useEffect(() => {
    const controller = new AbortController();

    const fetchData = async () => {
        try {
            const response = await fetch('/api/data', {
                signal: controller.signal
            });
            const data = await response.json();
            setData(data);
        } catch (err) {
            if (err.name !== 'AbortError') {
                setError(err.message);
            }
        }
    };

    fetchData();

    return () => controller.abort();
}, []);

// ==================== TIMER/INTERVAL ====================
function Timer() {
    const [seconds, setSeconds] = useState(0);

    useEffect(() => {
        const interval = setInterval(() => {
```

```
            setSeconds(prev => prev + 1);
        }, 1000);

        // Cleanup on unmount
        return () => clearInterval(interval);
    }, []);

    return <div>Seconds: {seconds}</div>;
}

// ==================== EVENT LISTENERS ====================
function WindowSize() {
    const [size, setSize] = useState({
        width: window.innerWidth,
        height: window.innerHeight
    });

    useEffect(() => {
        const handleResize = () => {
            setSize({
                width: window.innerWidth,
                height: window.innerHeight
            });
        };

        window.addEventListener('resize', handleResize);

        return () => window.removeEventListener('resize', handleResize);
    }, []);

    return <div>{size.width} x {size.height}</div>;
}

// ==================== LOCAL STORAGE ====================
function usePersistentState(key, defaultValue) {
    const [state, setState] = useState(() => {
        const saved = localStorage.getItem(key);
        return saved ? JSON.parse(saved) : defaultValue;
    });

    useEffect(() => {
        localStorage.setItem(key, JSON.stringify(state));
    }, [key, state]);

    return [state, setState];
}

// ==================== DOCUMENT TITLE ====================
useEffect(() => {
    document.title = `You have ${count} messages`;
}, [count]);
```

```
// ==================== DEPENDENCY GOTCHAS ====================

// Objects/arrays create new references each render
useEffect(() => {
    fetchData(options);
}, [options]); // ⚠️ Triggers every render if options is inline object

// Solution 1: Move inside useEffect
useEffect(() => {
    const options = { limit: 10 };
    fetchData(options);
}, []);

// Solution 2: useMemo for objects
const options = useMemo(() => ({ limit }), [limit]);
useEffect(() => {
    fetchData(options);
}, [options]);

// Solution 3: Primitive dependencies
useEffect(() => {
    fetchData({ limit: pageLimit });
}, [pageLimit]);
```

5.7 useContext Hook

> ***Definition:*** *`useContext` is a Hook that allows consuming context values without prop drilling. Context provides a way to share values (theme, auth state, language) between components at any nesting level without explicitly passing props through every component.*

```
import { createContext, useContext, useState } from 'react';

// ==================== 1. CREATE CONTEXT ====================
const ThemeContext = createContext();

// ==================== 2. CREATE PROVIDER COMPONENT ====================
function ThemeProvider({ children }) {
    const [theme, setTheme] = useState('light');

    const toggleTheme = () => {
        setTheme(prev => prev === 'light' ? 'dark' : 'light');
    };

    // Value object with all state and functions to share
    const value = {
        theme,
        toggleTheme,
        isDark: theme === 'dark'
    };
```

```
    return (
        <ThemeContext.Provider value={value}>
            {children}
        </ThemeContext.Provider>
    );
}

// ==================== 3. CUSTOM HOOK (Recommended) ====================
function useTheme() {
    const context = useContext(ThemeContext);
    if (context === undefined) {
        throw new Error('useTheme must be used within a ThemeProvider');
    }
    return context;
}

// ==================== 4. USE IN COMPONENTS ====================
function ThemeToggle() {
    const { theme, toggleTheme } = useTheme();

    return (
        <button onClick={toggleTheme}>
            Current: {theme}
        </button>
    );
}

function ThemedBox() {
    const { isDark } = useTheme();

    return (
        <div style={{
            background: isDark ? '#333' : '#fff',
            color: isDark ? '#fff' : '#333',
            padding: '20px'
        }}>
            Themed content
        </div>
    );
}

// ==================== 5. WRAP APP WITH PROVIDER ====================
function App() {
    return (
        <ThemeProvider>
            <div>
                <ThemeToggle />
                <ThemedBox />
            </div>
        </ThemeProvider>
    );
}
```

```
// ==================== AUTH CONTEXT EXAMPLE ====================
const AuthContext = createContext();

function AuthProvider({ children }) {
    const [user, setUser] = useState(null);
    const [loading, setLoading] = useState(true);

    useEffect(() => {
        // Check for saved session
        const token = localStorage.getItem('token');
        if (token) {
            verifyToken(token).then(setUser).finally(() => setLoading(false));
        } else {
            setLoading(false);
        }
    }, []);

    const login = async (email, password) => {
        const response = await fetch('/api/login', {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({ email, password })
        });
        const data = await response.json();
        localStorage.setItem('token', data.token);
        setUser(data.user);
        return data;
    };

    const logout = () => {
        localStorage.removeItem('token');
        setUser(null);
    };

    return (
        <AuthContext.Provider value={{ user, login, logout, loading }}>
            {children}
        </AuthContext.Provider>
    );
}

function useAuth() {
    const context = useContext(AuthContext);
    if (!context) {
        throw new Error('useAuth must be used within AuthProvider');
    }
    return context;
}

// Protected Route Component
function ProtectedRoute({ children }) {
```

```
    const { user, loading } = useAuth();

    if (loading) return <div>Loading...</div>;
    if (!user) return <Navigate to="/login" />;

    return children;
}

// ==================== MULTIPLE CONTEXTS ====================
function App() {
    return (
        <AuthProvider>
            <ThemeProvider>
                <LanguageProvider>
                    <Router>
                        <AppContent />
                    </Router>
                </LanguageProvider>
            </ThemeProvider>
        </AuthProvider>
    );
}
```

5.8 useRef Hook

> ***Definition:*** *`useRef` returns a mutable object with a `.current` property that persists across renders without causing re-renders when changed. It's commonly used to access DOM elements directly, store mutable values, and hold previous values.*

```
import { useRef, useState, useEffect } from 'react';

// ==================== DOM ELEMENT ACCESS ====================
function TextInput() {
    const inputRef = useRef(null);

    // Focus on mount
    useEffect(() => {
        inputRef.current.focus();
    }, []);

    const handleClick = () => {
        inputRef.current.focus();
        inputRef.current.select();
    };

    return (
        <div>
            <input ref={inputRef} type="text" />
            <button onClick={handleClick}>Focus Input</button>
        </div>
```

```
    );
}

// ==================== STORE MUTABLE VALUE ====================
// (doesn't trigger re-render when changed)
function Timer() {
    const [count, setCount] = useState(0);
    const intervalRef = useRef(null);

    const startTimer = () => {
        if (intervalRef.current) return; // Already running

        intervalRef.current = setInterval(() => {
            setCount(prev => prev + 1);
        }, 1000);
    };

    const stopTimer = () => {
        clearInterval(intervalRef.current);
        intervalRef.current = null;
    };

    const resetTimer = () => {
        stopTimer();
        setCount(0);
    };

    // Cleanup on unmount
    useEffect(() => {
        return () => clearInterval(intervalRef.current);
    }, []);

    return (
        <div>
            <p>Count: {count}</p>
            <button onClick={startTimer}>Start</button>
            <button onClick={stopTimer}>Stop</button>
            <button onClick={resetTimer}>Reset</button>
        </div>
    );
}

// ==================== PREVIOUS VALUE ====================
function Counter() {
    const [count, setCount] = useState(0);
    const prevCountRef = useRef();

    useEffect(() => {
        prevCountRef.current = count;
    });

    const prevCount = prevCountRef.current;
```

```
    return (
        <div>
            <p>Now: {count}, Before: {prevCount}</p>
            <button onClick={() => setCount(c => c + 1)}>+</button>
        </div>
    );
}

// Custom hook for previous value
function usePrevious(value) {
    const ref = useRef();
    useEffect(() => {
        ref.current = value;
    });
    return ref.current;
}

// ==================== CALLBACK REF ====================
function MeasureExample() {
    const [height, setHeight] = useState(0);

    const measuredRef = useCallback(node => {
        if (node !== null) {
            setHeight(node.getBoundingClientRect().height);
        }
    }, []);

    return (
        <div ref={measuredRef}>
            Height: {height}px
        </div>
    );
}

// ==================== FORWARD REF ====================
// Expose ref to parent component
const FancyInput = forwardRef((props, ref) => {
    return <input ref={ref} className="fancy" {...props} />;
});

function Parent() {
    const inputRef = useRef();

    return (
        <div>
            <FancyInput ref={inputRef} />
            <button onClick={() => inputRef.current.focus()}>
                Focus
            </button>
        </div>
```

```
    );
}
```

5.9 useReducer Hook

> ***Definition:*** *`useReducer` is a Hook for managing complex state logic using a reducer function (similar to Redux). It's an alternative to `useState` when state transitions are complex, depend on previous state, or when you have multiple sub-values.*

```
import { useReducer } from 'react';

// ==================== BASIC EXAMPLE ====================

// Define initial state
const initialState = { count: 0 };

// Define reducer function
function reducer(state, action) {
    switch (action.type) {
        case 'INCREMENT':
            return { count: state.count + 1 };
        case 'DECREMENT':
            return { count: state.count - 1 };
        case 'RESET':
            return initialState;
        case 'SET':
            return { count: action.payload };
        default:
            throw new Error(`Unknown action: ${action.type}`);
    }
}

function Counter() {
    const [state, dispatch] = useReducer(reducer, initialState);

    return (
        <div>
            <p>Count: {state.count}</p>
            <button onClick={() => dispatch({ type: 'INCREMENT' })}>+</button>
            <button onClick={() => dispatch({ type: 'DECREMENT' })}>-</button>
            <button onClick={() => dispatch({ type: 'RESET' })}>Reset</button>
            <button onClick={() => dispatch({ type: 'SET', payload: 100 })}>
                Set to 100
            </button>
        </div>
    );
}

// ==================== COMPLEX STATE ====================
```

```
const initialState = {
    todos: [],
    filter: 'all',
    loading: false,
    error: null
};

function todoReducer(state, action) {
    switch (action.type) {
        case 'SET_LOADING':
            return { ...state, loading: action.payload };

        case 'SET_ERROR':
            return { ...state, error: action.payload, loading: false };

        case 'SET_TODOS':
            return { ...state, todos: action.payload, loading: false };

        case 'ADD_TODO':
            return {
                ...state,
                todos: [...state.todos, action.payload]
            };

        case 'TOGGLE_TODO':
            return {
                ...state,
                todos: state.todos.map(todo =>
                    todo.id === action.payload
                        ? { ...todo, completed: !todo.completed }
                        : todo
                )
            };

        case 'DELETE_TODO':
            return {
                ...state,
                todos: state.todos.filter(todo => todo.id !== action.payload)
            };

        case 'SET_FILTER':
            return { ...state, filter: action.payload };

        case 'CLEAR_COMPLETED':
            return {
                ...state,
                todos: state.todos.filter(todo => !todo.completed)
            };

        default:
            return state;
    }
```

```
}

function TodoApp() {
    const [state, dispatch] = useReducer(todoReducer, initialState);

    // Async action
    const fetchTodos = async () => {
        dispatch({ type: 'SET_LOADING', payload: true });
        try {
            const response = await fetch('/api/todos');
            const data = await response.json();
            dispatch({ type: 'SET_TODOS', payload: data });
        } catch (error) {
            dispatch({ type: 'SET_ERROR', payload: error.message });
        }
    };

    const addTodo = (text) => {
        dispatch({
            type: 'ADD_TODO',
            payload: { id: Date.now(), text, completed: false }
        });
    };

    // Filter todos based on state.filter
    const filteredTodos = state.todos.filter(todo => {
        if (state.filter === 'active') return !todo.completed;
        if (state.filter === 'completed') return todo.completed;
        return true;
    });

    return (
        <div>
            {state.loading && <p>Loading...</p>}
            {state.error && <p>Error: {state.error}</p>}
            {/* Render todos */}
        </div>
    );
}

// ==================== LAZY INITIALIZATION ====================
const init = (initialCount) => {
    return { count: initialCount };
};

const [state, dispatch] = useReducer(reducer, 0, init);
```

5.10 useMemo & useCallback

> ***Definition:*** `useMemo` *memoizes computed values to avoid expensive recalculations on every render.* `useCallback` *memoizes functions to maintain referential equality, preventing unnecessary child re-renders when passed as props. Both accept a dependency array.*

```
import { useMemo, useCallback, useState, memo } from 'react';

// ==================== useMemo ====================
// Memoize expensive computations

function ExpensiveComponent({ items, filter }) {
    // Only recalculates when items or filter changes
    const filteredItems = useMemo(() => {
        console.log('Filtering items...');
        return items.filter(item =>
            item.name.toLowerCase().includes(filter.toLowerCase())
        );
    }, [items, filter]);

    // Expensive calculation
    const statistics = useMemo(() => {
        console.log('Calculating stats...');
        return {
            total: items.length,
            average: items.reduce((a, b) => a + b.price, 0) / items.length,
            max: Math.max(...items.map(i => i.price))
        };
    }, [items]);

    return (
        <div>
            <p>Stats: {JSON.stringify(statistics)}</p>
            <ul>
                {filteredItems.map(item => (
                    <li key={item.id}>{item.name}</li>
                ))}
            </ul>
        </div>
    );
}

// ==================== useCallback ====================
// Memoize functions

function ParentComponent() {
    const [count, setCount] = useState(0);
    const [items, setItems] = useState([]);

    // Without useCallback: new function every render
    // Child re-renders unnecessarily
    const handleClick = () => {
        console.log('Clicked');
```

```
    };

    // With useCallback: same function reference
    const handleClickMemoized = useCallback(() => {
        console.log('Clicked');
    }, []); // Empty deps = never recreated

    // With dependencies
    const handleAddItem = useCallback((item) => {
        setItems(prev => [...prev, item]);
    }, []); // Recreated only if setItems changes (never)

    // Callback that uses state
    const handleDelete = useCallback((id) => {
        setItems(prev => prev.filter(item => item.id !== id));
    }, []);

    return (
        <div>
            <ChildComponent
                onClick={handleClickMemoized}
                onAddItem={handleAddItem}
            />
        </div>
    );
}

// ==================== React.memo ====================
// Memoize component (only re-renders if props change)

const ChildComponent = memo(function ChildComponent({ onClick, onAddItem }) {
    console.log('Child rendered');
    return (
        <div>
            <button onClick={onClick}>Click</button>
        </div>
    );
});

// Custom comparison
const ChildComponent = memo(
    function ChildComponent({ user, onClick }) {
        return <div>{user.name}</div>;
    },
    (prevProps, nextProps) => {
        // Return true if props are equal (skip re-render)
        return prevProps.user.id === nextProps.user.id;
    }
);

// ==================== WHEN TO USE ====================
```

```
// ✅ Good use cases
const sortedItems = useMemo(() =>
    [...items].sort((a, b) => a.name.localeCompare(b.name)),
    [items]
);

const handleSubmit = useCallback((data) => {
    api.submit(data);
}, []);

// ❌ Probably unnecessary (cheap operations)
const doubled = useMemo(() => count * 2, [count]);
const message = useMemo(() => `Hello ${name}`, [name]);
```

5.11 Custom Hooks

> ***Definition:*** *Custom Hooks are reusable functions that encapsulate stateful logic using built-in Hooks. They must start with "use" and allow sharing logic between components without changing the component hierarchy. They follow the rules of Hooks.*

```
import { useState, useEffect, useCallback } from 'react';

// ==================== useFetch ====================
function useFetch(url) {
    const [data, setData] = useState(null);
    const [loading, setLoading] = useState(true);
    const [error, setError] = useState(null);

    useEffect(() => {
        const controller = new AbortController();

        const fetchData = async () => {
            try {
                setLoading(true);
                setError(null);

                const response = await fetch(url, {
                    signal: controller.signal
                });

                if (!response.ok) {
                    throw new Error(`HTTP error! status: ${response.status}`);
                }

                const result = await response.json();
                setData(result);
            } catch (err) {
                if (err.name !== 'AbortError') {
                    setError(err.message);
                }
```

```
            } finally {
                setLoading(false);
            }
        };

        fetchData();

        return () => controller.abort();
    }, [url]);

    return { data, loading, error };
}

// Usage
function UserList() {
    const { data: users, loading, error } = useFetch('/api/users');

    if (loading) return <p>Loading...</p>;
    if (error) return <p>Error: {error}</p>;

    return (
        <ul>
            {users.map(user => <li key={user.id}>{user.name}</li>)}
        </ul>
    );
}

// ==================== useLocalStorage ====================
function useLocalStorage(key, initialValue) {
    const [storedValue, setStoredValue] = useState(() => {
        try {
            const item = window.localStorage.getItem(key);
            return item ? JSON.parse(item) : initialValue;
        } catch (error) {
            console.error(error);
            return initialValue;
        }
    });

    const setValue = useCallback((value) => {
        try {
            const valueToStore = value instanceof Function
                ? value(storedValue)
                : value;
            setStoredValue(valueToStore);
            window.localStorage.setItem(key, JSON.stringify(valueToStore));
        } catch (error) {
            console.error(error);
        }
    }, [key, storedValue]);

    return [storedValue, setValue];
```

```
}

// Usage
const [theme, setTheme] = useLocalStorage('theme', 'dark');

// ==================== useToggle ====================
function useToggle(initialValue = false) {
    const [value, setValue] = useState(initialValue);

    const toggle = useCallback(() => {
        setValue(prev => !prev);
    }, []);

    const setTrue = useCallback(() => setValue(true), []);
    const setFalse = useCallback(() => setValue(false), []);

    return [value, toggle, setTrue, setFalse];
}

// Usage
const [isOpen, toggleOpen, open, close] = useToggle(false);

// ==================== useDebounce ====================
function useDebounce(value, delay = 500) {
    const [debouncedValue, setDebouncedValue] = useState(value);

    useEffect(() => {
        const timer = setTimeout(() => {
            setDebouncedValue(value);
        }, delay);

        return () => clearTimeout(timer);
    }, [value, delay]);

    return debouncedValue;
}

// Usage - Search with debounce
function SearchComponent() {
    const [searchTerm, setSearchTerm] = useState('');
    const debouncedSearch = useDebounce(searchTerm, 300);

    useEffect(() => {
        if (debouncedSearch) {
            // API call only happens 300ms after user stops typing
            searchAPI(debouncedSearch);
        }
    }, [debouncedSearch]);

    return (
        <input
            value={searchTerm}
```

```
            onChange={e => setSearchTerm(e.target.value)}
            placeholder="Search..."
        />
    );
}

// ==================== useWindowSize ====================
function useWindowSize() {
    const [size, setSize] = useState({
        width: window.innerWidth,
        height: window.innerHeight
    });

    useEffect(() => {
        const handleResize = () => {
            setSize({
                width: window.innerWidth,
                height: window.innerHeight
            });
        };

        window.addEventListener('resize', handleResize);
        return () => window.removeEventListener('resize', handleResize);
    }, []);

    return size;
}

// ==================== useClickOutside ====================
function useClickOutside(ref, handler) {
    useEffect(() => {
        const listener = (event) => {
            if (!ref.current || ref.current.contains(event.target)) {
                return;
            }
            handler(event);
        };

        document.addEventListener('mousedown', listener);
        document.addEventListener('touchstart', listener);

        return () => {
            document.removeEventListener('mousedown', listener);
            document.removeEventListener('touchstart', listener);
        };
    }, [ref, handler]);
}

// Usage
function Dropdown() {
    const ref = useRef();
    const [isOpen, setIsOpen] = useState(false);
```

```
    useClickOutside(ref, () => setIsOpen(false));

    return (
        <div ref={ref}>
            <button onClick={() => setIsOpen(!isOpen)}>Toggle</button>
            {isOpen && <div className="menu">Menu items</div>}
        </div>
    );
}
```

5.12 Forms & Events

> ***Definition:*** *React handles forms through controlled components where form input values are controlled by React state. Events in React use camelCase naming and pass synthetic events that wrap native browser events for cross-browser compatibility.*

```
import { useState } from 'react';

// ==================== CONTROLLED INPUTS ====================
function LoginForm() {
    const [formData, setFormData] = useState({
        email: '',
        password: '',
        rememberMe: false
    });
    const [errors, setErrors] = useState({});

    const handleChange = (e) => {
        const { name, value, type, checked } = e.target;
        setFormData(prev => ({
            ...prev,
            [name]: type === 'checkbox' ? checked : value
        }));
    };

    const validate = () => {
        const newErrors = {};
        if (!formData.email) newErrors.email = 'Email required';
        if (!formData.email.includes('@')) newErrors.email = 'Invalid email';
        if (formData.password.length < 6) {
            newErrors.password = 'Password must be 6+ chars';
        }
        setErrors(newErrors);
        return Object.keys(newErrors).length === 0;
    };

    const handleSubmit = (e) => {
        e.preventDefault();  // Prevent page reload
```

```
        if (validate()) {
            console.log('Submit:', formData);
            // API call here
        }
    };

    return (
        <form onSubmit={handleSubmit}>
            <div>
                <input
                    type="email"
                    name="email"
                    value={formData.email}
                    onChange={handleChange}
                    placeholder="Email"
                />
                {errors.email && <span className="error">{errors.email}</span>}
            </div>

            <div>
                <input
                    type="password"
                    name="password"
                    value={formData.password}
                    onChange={handleChange}
                    placeholder="Password"
                />
                {errors.password && <span className="error">{errors.password}</span>}
            </div>

            <div>
                <label>
                    <input
                        type="checkbox"
                        name="rememberMe"
                        checked={formData.rememberMe}
                        onChange={handleChange}
                    />
                    Remember me
                </label>
            </div>

            <button type="submit">Login</button>
        </form>
    );
}

// ==================== SELECT & TEXTAREA ====================
function ProfileForm() {
    const [profile, setProfile] = useState({
        name: '',
        bio: '',
```

```
        role: 'user'
    });

    return (
        <form>
            <input
                type="text"
                value={profile.name}
                onChange={e => setProfile({...profile, name: e.target.value})}
            />

            <textarea
                value={profile.bio}
                onChange={e => setProfile({...profile, bio: e.target.value})}
                rows={4}
            />

            <select
                value={profile.role}
                onChange={e => setProfile({...profile, role: e.target.value})}
            >
                <option value="user">User</option>
                <option value="admin">Admin</option>
                <option value="moderator">Moderator</option>
            </select>
        </form>
    );
}

// ==================== EVENT HANDLING ====================
function EventExamples() {
    // Click event
    const handleClick = (e) => {
        console.log('Clicked!', e.currentTarget);
    };

    // With parameters
    const handleItemClick = (id) => (e) => {
        console.log('Item clicked:', id);
    };

    // Keyboard events
    const handleKeyDown = (e) => {
        if (e.key === 'Enter') {
            console.log('Enter pressed');
        }
        if (e.key === 'Escape') {
            console.log('Escape pressed');
        }
    };

    // Prevent default
```

```
    const handleLinkClick = (e) => {
        e.preventDefault();
        console.log('Link clicked but not navigated');
    };

    // Stop propagation
    const handleInnerClick = (e) => {
        e.stopPropagation();
        console.log('Only inner, not outer');
    };

    return (
        <div>
            <button onClick={handleClick}>Click me</button>
            <button onClick={handleItemClick(123)}>Item 123</button>
            <input onKeyDown={handleKeyDown} />
            <a href="/page" onClick={handleLinkClick}>Link</a>

            <div onClick={() => console.log('Outer')}>
                <button onClick={handleInnerClick}>Inner</button>
            </div>
        </div>
    );
}
```

5.13 Conditional Rendering

> ***Definition:*** *Conditional rendering in React works the same way as JavaScript conditions. You can use if statements, ternary operators, or logical && to conditionally include elements in the output based on component state or props.*

```
function ConditionalExamples({ user, messages, isLoading, error }) {
    // ==================== IF/ELSE (Early return) ====================
    if (isLoading) {
        return <LoadingSpinner />;
    }

    if (error) {
        return <ErrorMessage message={error} />;
    }

    if (!user) {
        return <LoginPrompt />;
    }

    // ==================== TERNARY OPERATOR ====================
    return (
        <div>
            {user.isAdmin ? <AdminPanel /> : <UserPanel />}

            <div className={isActive ? 'active' : 'inactive'}>
```

```
                Status
            </div>
        </div>
    );

    // ==================== LOGICAL && (Short-circuit) ====================
    return (
        <div>
            {user && <UserProfile user={user} />}
            {messages.length > 0 && <Badge count={messages.length} />}
            {isAdmin && <AdminControls />}
        </div>
    );

    // ⚠️ Watch out for falsy values with &&
    // ❌ This renders "0" when count is 0
    {count && <span>{count}</span>}

    // ✅ Correct way
    {count > 0 && <span>{count}</span>}
    {count !== 0 && <span>{count}</span>}

    // ==================== NULL FOR NOTHING ====================
    return showMessage ? <Message /> : null;

    // ==================== SWITCH IN RENDER ====================
    const renderContent = () => {
        switch (status) {
            case 'loading':
                return <Loading />;
            case 'error':
                return <Error />;
            case 'success':
                return <Success />;
            default:
                return null;
        }
    };

    return <div>{renderContent()}</div>;

    // ==================== OBJECT MAPPING ====================
    const statusComponents = {
        loading: <Loading />,
        error: <Error />,
        success: <Success />
    };

    return <div>{statusComponents[status]}</div>;
}
```

5.14 Lists & Keys

> ***Definition:*** *React uses the* `map()` *function to render lists of elements. Each list item needs a unique* `key` *prop to help React identify which items have changed, been added, or removed. Keys should be stable, unique among siblings, and ideally from your data.*

```
function ListExamples() {
    const items = [
        { id: 1, name: 'Apple', price: 1.5 },
        { id: 2, name: 'Banana', price: 0.75 },
        { id: 3, name: 'Orange', price: 2.0 }
    ];

    // ==================== BASIC LIST ====================
    return (
        <ul>
            {items.map(item => (
                <li key={item.id}>{item.name}</li>
            ))}
        </ul>
    );

    // ==================== WITH COMPONENT ====================
    return (
        <div>
            {items.map(item => (
                <ProductCard
                    key={item.id}
                    name={item.name}
                    price={item.price}
                />
            ))}
        </div>
    );

    // ==================== SPREAD PROPS ====================
    return (
        <div>
            {items.map(item => (
                <ProductCard key={item.id} {...item} />
            ))}
        </div>
    );

    // ==================== WITH INDEX (Last resort) ====================
    // Only use index if items have no unique ID and list is static
    const staticItems = ['Home', 'About', 'Contact'];

    return (
        <nav>
            {staticItems.map((item, index) => (
```

```
                <a key={index} href={`/${item.toLowerCase()}`}>
                    {item}
                </a>
            ))}
        </nav>
    );

    // ⚠️ Never use index as key for dynamic lists!
    // It causes bugs with reordering, adding, removing
}

// ==================== KEY RULES ====================
// ✅ Use unique, stable IDs from your data
// ✅ Keys only need to be unique among siblings
// ❌ Don't use array index for dynamic lists
// ❌ Don't use random values (Math.random())
// ❌ Don't generate keys during render
```

5.15 Virtual DOM

> ***Definition:*** *The Virtual DOM is an in-memory representation of the real DOM elements. When state changes, React creates a new virtual DOM tree, compares it with the previous one using a "diffing" algorithm, and efficiently updates only the changed portions in the actual DOM (reconciliation).*

```
How Virtual DOM Works:

1. State Change
   └── Component calls setState()

2. New Virtual DOM
   └── React creates new virtual DOM tree

3. Diffing
   └── Compare new vs old virtual DOM
   └── Uses heuristics for O(n) complexity
   └── Key prop helps identify list items

4. Reconciliation
   └── Calculate minimum DOM operations
   └── Batch updates for performance

5. Real DOM Update
   └── Apply only necessary changes
```

Key Points for Interview:

- Virtual DOM is a JavaScript object representation of DOM
- Changes are batched for performance
- Diffing algorithm works in O(n) time
- Keys help React identify list item changes
- React Fiber (React 16+) allows incremental rendering

Next: [Chapter 6 - Quick Reference & MCQs]